PNAS RESEARCH ARTICLE | APPLIED MATHEMATICS OPEN ACCESS

Fast automated adjoints for spectral PDE solvers

Calum S. Skene[a,b,1,2] and Keaton J. Burns[c,d,1,2]

Affiliations are included on p. 9.

Edited by Leslie Greengard, Flatiron Institute, New York, NY; received October 23, 2025; accepted March 9, 2026

We present an automated procedure for computing model gradients for partial differential equation (PDE) solvers built on sparse spectral methods, a broad class of numerical techniques widely used in the study of fluid dynamics, continuum mechanics, waves, and pattern formation across disciplines. Our approach applies reverse-mode automatic differentiation to symbolic graph representations of PDEs and efficiently constructs adjoint solvers that retain the speed and flexibility of modern Fourier and polynomial spectral methods. We demonstrate the advantages of this approach with a comprehensive implementation in the open-source Dedalus framework. This work uniquely provides a differentiable spectral solver that supports a broad class of equations, geometries, and boundary conditions, and runs efficiently in parallel. It enables users to compute gradients and perform PDE-based optimization for a wide range of time-dependent and nonlinear models with minimal additional code. We demonstrate this system's capabilities using canonical problems from the literature, showing both strong performance and practical utility for a wide variety of optimization tasks. By integrating automatic adjoints into a flexible solver, our work enables researchers to perform sensitivity analyses in spectral simulations with ease and efficiency.

automatic differentiation | PDE-based optimization | spectral methods | computational fluid dynamics | adjoint sensitivity analysis

Significance

Scientific computing increasingly requires both accurate simulations of physical systems as well as optimizing them over many parameters for tasks such as inverse modeling, system control, and variational data assimilation. Adjoint methods enable these optimizations by efficiently computing model gradients, but they traditionally require laborious and problem-specific derivations. Here, we present a framework that automatically generates discrete adjoints of partial differential equation (PDE) solvers based on spectral discretizations. Spectral methods are widely used for their speed and accuracy, yet they have lacked a flexible and automated approach for performant adjoint computations. Our framework unlocks fast and scalable sensitivity analysis for spectral simulations, enabling efficient PDE-based optimization across applications including fluid dynamics, plasma physics, biology, and the Earth sciences.

Numerical modeling has become a ubiquitous tool across every discipline of science and engineering. However, systematically using forward models to enhance scientific understanding often requires costly optimizations over model inputs and parameters. Adjoint state methods provide an efficient and systematic way to compute the model gradients needed for such model-based optimization. Thus, the ability to compute adjoints forms the foundation of many scientific studies that would otherwise be infeasible. Indeed, the recent rise of machine learning and artificial intelligence as a transformative technology would not have been possible without adjoint methods, which form the basis of the backpropagation algorithms needed to train models. Beyond machine learning, adjoint methods have a rich history in scientific computing. They are an indispensable tool in nonmodal stability theory (1, 2), parametric sensitivity analysis (3, 4), and weather forecasting via four-dimensional data assimilation (4D-Var) (5, 6). Adjoint techniques have also been applied in countless optimization settings—for example, to design aerodynamic structures (7), mitigate thermoacoustic instabilities (8), optimize stellarator designs for nuclear fusion reactors (9, 10), and solve inverse problems in geophysics (11), among many other applications.

Despite their evident utility, adjoint-based computations remain underutilized in many fields, largely due to the difficulty of computing the adjoint operator, which is often a tedious and error-prone process. For numerical solvers of partial differential equations (PDEs) (e.g., fluid flow or electromagnetics), there are two main strategies for obtaining adjoints. One approach is to analytically derive an adjoint system by integrating the governing equations by parts in both space and time; the resulting continuous adjoint equations are then discretized and solved numerically. Alternatively, the PDE can be discretized first, and then the adjoint of the resulting discrete system can be computed using tools like automatic differentiation (AD). These are known as the continuous and discrete adjoint approaches, respectively, each with their own advantages and disadvantages (see refs. 12 and 13 for detailed discussions).

Several open-source simulation codes now include built-in adjoint functionality (often termed "differentiable" codes), using a mix of continuous and discrete approaches. This includes SU^2 (14), FEniCS (15), Firedrake (16), Φ_{Flow} (17), simsopt (10), OpenFOAM (18), Exponax (19), JaxFluids (20, 21), JAX-CFD (22, 23), Trixi.jl (24), and Julia's SciML ecosystem (25). The majority of these PDE solvers focus on

Author contributions: C.S.S. and K.J.B. designed research; performed research; analyzed data; and wrote the paper.

The authors declare no competing interest.

This article is a PNAS Direct Submission.

[1]C.S.S. and K.J.B. contributed equally to this work.

[2]To whom correspondence may be addressed. Email: cskene3@ed.ac.uk or kjburns@mit.edu.

This article contains supporting information online at https://www.pnas.org/lookup/suppl/doi:10.1073/pnas.2530440123/-/DCSupplemental.

Published April 10, 2026.

specific equation sets and are tied to particular applications or geometries. Notably, FEniCS and Firedrake are different in that they are high-level finite-element libraries designed for a wide variety of models. To use these packages, the governing equations must be specified in variational (weak) form via the "unified form language" (26), which are then discretized by the library. Adjoint computations are executed automatically using the dolfin-adjoint framework (27, 28). This approach has enabled numerous adjoint-based studies across a range of PDE applications (e.g., refs. 29 and 30).

While finite-element methods excel for complex geometries, global spectral methods are a popular and complementary alternative for problems in simpler geometries, such as turbulent fluid flow in a box or a sphere. These methods offer exponential accuracy as resolution is increased, and they facilitate efficient elliptic solves for differential-algebraic equations as well as implicit time-stepping for stiff problems (31). Spectral methods are widely used in fundamental research and underpin the world's largest turbulence simulations (32) and state-of-the-art weather forecasting models (33). The JAX-CFD code includes support for automatic differentiation of Fourier spectral solvers for incompressible hydrodynamics, but does not support high-order timestepping, nonperiodic boundary conditions, steady-state simulations, or distributed-memory parallelism. Beyond classical Fourier methods, recent developments in sparse polynomial spectral methods have extended Fourier-like speed and accuracy to simple geometries in Cartesian (34), polar (35), and spherical coordinates (36). These advances have enabled cutting-edge simulations of biophysical, geophysical, and astrophysical flows (e.g., refs. 37–39).

Here, we present a numerical approach to automatically generate discrete adjoints for the entire family of fast global spectral methods, implemented in the open-source Dedalus framework (40). Like FEniCS and Firedrake, Dedalus is a high-level library that discretizes and solves user-defined PDEs. However, Dedalus allows users to specify systems of equations in strong form via a simple interface that is both flexible and accessible to computational scientists across many disciplines. Dedalus includes many spectral bases and has solver paths for eigenvalue problems (EVPs), linear boundary value problems (LBVPs), nonlinear boundary value problems (NLBVPs), and initial value problems (IVPs). Dedalus is written in Python, with compiled extensions for performance-critical routines, and it automatically handles distributed-memory parallelism via MPI. These features allow Dedalus simulations to be easily prototyped on a laptop and then run at scale on high-performance computing clusters.

Our procedure applies AD to the high-level equation representation in Dedalus and fuses discrete adjoint routines for the underlying solver components. By exploiting the modularity of the codebase, we have made Dedalus differentiable without needing to rewrite the library to be compatible with any particular AD toolchain. However, our implementation can still easily interface with external optimization libraries [e.g., Manopt (41)] and integrators [e.g., PETSc (42)], and allows combining sparse spectral solvers with machine learning frameworks [e.g., PyTorch (43)], which inherently require differentiable simulators for training.

In this article, we demonstrate the capabilities of our adjoint framework by using Dedalus to solve several representative optimization problems drawn from a variety of scientific domains. These examples highlight the efficiency, flexibility, and ease of use of our differentiable extension of Dedalus. By working with discrete adjoints, our method automatically handles any boundary conditions, constraints, and gauge choices, eliminating the difficulties these pose for continuous-adjoint approaches. Importantly, in all these examples, the gradient computation requires minimal user intervention: Only a few additional lines of code are needed to obtain the gradient of any cost functional evaluated on a Dedalus solution. Overall, our work makes adjoint-based optimization readily accessible to scientists using spectral methods across many workflows and applications.

Spectral Adjoints

Continuous vs. Discrete. The fundamental definition of an adjoint is mathematically simple: An adjoint operator $\mathcal{A}^\dagger$ to a linear operator $\mathcal{A}$ is one that satisfies $\langle x, \mathcal{A}y\rangle = \langle \mathcal{A}^\dagger x, y\rangle$ for all x and y, where $\langle \cdot, \cdot \rangle$ is a suitably chosen inner product. Efficiently solving the adjoint of a numerical model's Jacobian yields the model's gradients. Continuous methods discretize the infinite dimensional adjoint, while discrete methods discretize the forward operator and then take its finite-dimensional adjoint.

The continuous adjoint has the advantage of producing the true adjoint of the governing equations. However, it generally does not provide the exact gradient of the discrete forward model (even if the forward model is well resolved), and deriving it—either manually or automatically—is difficult for constrained systems or complex boundary conditions. Implementing a continuous adjoint also requires writing a separate solver in addition to the original model. The discrete adjoint, on the other hand, yields the correct gradient for the discrete forward model. This advantage often translates into better convergence in outer-loop optimizations, although it can introduce instabilities when the forward model is underresolved (12, 25).

Several studies have manually implemented continuous adjoints for specific PDEs in Dedalus using its symbolic equation-entry interface. For example, adjoint methods have been used to explore the similarities between minimal seeds and instantons (44), to solve inverse problems (45), and to identify subcritical geodynamo solutions (46). In a recent study (47), continuous and discrete adjoints were compared across several Cartesian problems in Dedalus, highlighting the advantages of the discrete approach—especially when used with optimization methods that benefit from the discrete adjoint's superior convergence. In each of these cases, implementing the adjoint solver demanded significant user effort (analytical derivations or manual coding of the discrete adjoint), particularly for bounded domains and high-order time-stepping schemes.

Importantly, models implemented within AD frameworks [e.g., Enzyme (48), JAX (49), or Tapenade (50)] can automatically construct discrete adjoints directly from forward-model code, contributing to their growing popularity in recent years. However, AD toolchains often face limitations in programming-language interoperability and hardware support. Moreover, high-performance library routines (e.g., FFTs and linear algebra kernels) typically require user-defined custom derivatives or "chain rules," since explicitly providing Jacobian actions is often more efficient and numerically stable than tracing the full forward computation. For the highly structured operations common in high-order PDE solvers, such custom rules may be necessary for most of the computational pipeline, diminishing the benefits of end-to-end AD.

Here, we take the approach of leveraging the existing flexible, high-level code structures that enable Dedalus to forward model wide varieties of PDEs. We define adjoints of the various solver

types at the outer level, and use the code's internal computational graphs to perform a highly structured form of reverse-mode AD that combines the discrete adjoints of the low-level operator implementations. Our approach eliminates the barriers associated with manually deriving continuous adjoint systems and produces discrete adjoints without requiring modifications of the forward solver or relying on external AD libraries. This combination enables Dedalus to automatically compute the discrete adjoint for every available solver type, geometry, and time-stepping scheme in an efficient and platform-independent manner.

Sparse Spectral Methods. Spectral methods form finite-dimensional discretizations of PDEs by expanding the unknown solutions in a trial basis and projecting the equations against a test basis, enforcing the strong-form PDE via a weighted-residual method. Recent advances in the field have established optimal choices for the test and trial bases to produce maximally sparse discretizations for wide ranges of equations and domains (34–36, 51–53). Linear operators can be directly discretized into sparse (often narrowly banded) matrices, which are fast to apply or invert. Nonlinear terms can be efficiently evaluated on a collocation grid using fast transforms (e.g., FFTs). Boundary conditions and other constraints can be applied by modifying the discretized systems, or at the continuous level through the use of Lagrange multipliers and tau terms (54).

Solvers for wide classes of BVPs and IVPs can then be readily automated as series of sparse linear system solves against explicitly computed right-hand sides (RHSs); for instance, this encapsulates both Newton iterations for BVPs and mixed implicit-explicit (IMEX) timestepping for IVPs. For notational brevity, we will denote such a forward solution as a vector of solution coefficients $\mathbf{X}$ satisfying the equation $\mathbf{F}(\mathbf{X}, \mathbf{p}) = 0$, where $\mathbf{p}$ is a vector of problem parameters. In *SI Appendix*, we provide more details on how EVPs and IVPs are handled, but we will illustrate the process using this simple notation that more naturally matches the form of BVPs. The goal of the discrete adjoint program is to efficiently compute gradients of solution properties with respect to the parameters, and to do so by reusing the flexible and efficient computational motifs (sparse direct solvers and fast transforms) that enable the automated forward solution of PDEs with spectral methods.

Efficiently Calculating Gradients. Here, we summarize how discrete adjoints are used to obtain the derivative with respect to parameters $\mathbf{p}$—which could be forcing terms, initial conditions, or equation parameters—of a functional applied to the spectral solution of a PDE. We denote the target functional as $J(\tilde{\mathbf{X}}, \mathbf{p})$, where $\tilde{\mathbf{X}}$ is a generic vector in the finite-dimensional solution space. We seek to compute the derivative at a point $\tilde{\mathbf{X}} = \mathbf{X}(\mathbf{p})$ which solves the discretized PDE with the specified parameters, i.e., $\mathbf{F}(\mathbf{X}(\mathbf{p}), \mathbf{p}) = 0$. As a functional purely of the parameters, we define $\mathcal{J}(\mathbf{p}) = J(\mathbf{X}(\mathbf{p}), \mathbf{p})$. Directly differentiating this functional by the chain rule yields

$$\frac{\mathrm{d}\mathcal{J}}{\mathrm{d}\mathbf{p}} = \frac{\partial J}{\partial \mathbf{p}} + \frac{\partial J}{\partial \tilde{\mathbf{X}}} \cdot \frac{\mathrm{d}\mathbf{X}}{\mathrm{d}\mathbf{p}}, \qquad [1]$$

where the partial derivatives are evaluated at $(\mathbf{X}(\mathbf{p}), \mathbf{p})$. The difficulty in computing this gradient explicitly is the last term, $\mathrm{d}\mathbf{X}/\mathrm{d}\mathbf{p}$, since directly calculating this Jacobian by finite differences requires solving the forward equation for every element of $\mathbf{p}$, which becomes prohibitively expensive as the number of parameters increases.

To obtain the gradient with a tractable computational procedure, the adjoint state method approaches this as a constrained optimization problem (for a detailed review, see refs. 11 and 55). The associated Lagrangian is

$$\mathcal{L}(\tilde{\mathbf{X}}, \tilde{\mathbf{Y}}, \mathbf{p}) = J(\tilde{\mathbf{X}}, \mathbf{p}) - \langle \tilde{\mathbf{Y}}, \mathbf{F}(\tilde{\mathbf{X}}, \mathbf{p}) \rangle, \qquad [2]$$

where $\tilde{\mathbf{X}}$ and $\tilde{\mathbf{Y}}$ are generic vectors. Applying the chain rule directly to $\mathcal{L}$ gives

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathbf{p}} = \frac{\partial \mathcal{L}}{\partial \mathbf{p}} + \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{X}}} \cdot \frac{\partial \tilde{\mathbf{X}}}{\partial \mathbf{p}} + \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{Y}}} \cdot \frac{\partial \tilde{\mathbf{Y}}}{\partial \mathbf{p}}. \qquad [3]$$

By construction, the last term in this expression will be zero when evaluating the gradient at $\tilde{\mathbf{X}} = \mathbf{X}(\mathbf{p})$ because $(\partial \mathcal{L}/\partial \tilde{\mathbf{Y}})|_{\mathbf{X}} = -\mathbf{F}(\mathbf{X}, \mathbf{p}) = 0$. Calculating $\partial \mathcal{L}/\partial \tilde{\mathbf{X}}$ yields

$$\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{X}}} = \frac{\partial J}{\partial \tilde{\mathbf{X}}} - \left(\frac{\partial \mathbf{F}}{\partial \tilde{\mathbf{X}}}\right)^{\dagger} \tilde{\mathbf{Y}}, \qquad [4]$$

where † indicates the Hermitian adjoint. This term can be set to zero by choosing $\tilde{\mathbf{Y}} = \mathbf{Y}(\mathbf{p})$, where $\mathbf{Y}(\mathbf{p})$ satisfies the adjoint state equation,

$$\left(\frac{\partial \mathbf{F}}{\partial \tilde{\mathbf{X}}}\right)^{\dagger} \mathbf{Y}(\mathbf{p}) = \frac{\partial J}{\partial \tilde{\mathbf{X}}}, \qquad [5]$$

where again all partials are evaluated at $(\mathbf{X}(\mathbf{p}), \mathbf{p})$. Hence, by setting $\tilde{\mathbf{X}} = \mathbf{X}(\mathbf{p})$, $\tilde{\mathbf{Y}} = \mathbf{Y}(\mathbf{p})$, and by using the fact that $\mathcal{L}(\mathbf{X}(\mathbf{p}), \cdot, \mathbf{p}) = \mathcal{J}(\mathbf{p})$, we obtain

$$\frac{\mathrm{d}\mathcal{J}}{\mathrm{d}\mathbf{p}} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\mathbf{p}} = \frac{\partial \mathcal{L}}{\partial \mathbf{p}} = \frac{\partial J}{\partial \mathbf{p}} - \left\langle \mathbf{Y}, \frac{\partial \mathbf{F}}{\partial \mathbf{p}} \right\rangle. \qquad [6]$$

Eq. **6** is key to the adjoint method and shows that by first solving for $\mathbf{Y}(\mathbf{p})$ we can efficiently obtain the gradient of $\mathcal{J}$, since the dimension of the adjoint state is independent of the number of parameters.

To summarize, in the adjoint state method:

1. The forward problem is first solved to determine $\mathbf{X}(\mathbf{p})$.
2. The adjoint problem (Eq. **5**) is then solved for $\mathbf{Y}(\mathbf{p})$.
3. Finally, the right-hand side of Eq. **6** is computed to determine the gradient $\mathrm{d}\mathcal{J}/\mathrm{d}\mathbf{p}$.

The first (forward) step is already automated in Dedalus using optimally sparse spectral methods. We have now automated the second and third (adjoint) steps, enabling automatic PDE-based optimization through a simple user interface. The principal technical requirements are

- Implementing fast direct solvers for the Jacobian adjoints appearing on the left-hand side of Eq. **5**. This is achieved by reusing sparse factorizations from the forward solver stages.
- Adding fast evaluations of the derivative terms on the right-hand side of Eqs. **5** and **6**. This is achieved by implementing matrix-free vector-Jacobian products (VJPs) for arbitrary nonlinear operator graphs, via a custom high-level AD system that leverages fast discrete adjoints for the individual operator components.

The workflow of the adjoint looping process using these tools is illustrated in Fig. 1. More details about the technical implementations are provided in *Materials and Methods* and *SI Appendix*.

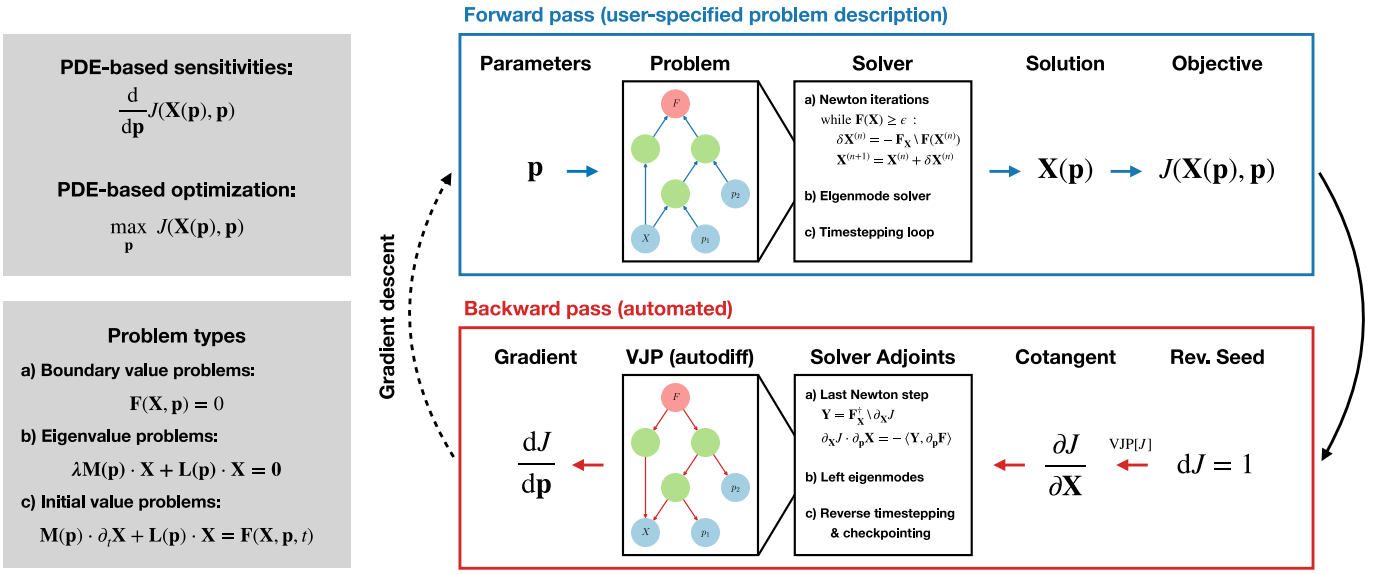

Fig. 1. Depiction of the adjoint-looping process to perform efficient PDE-based optimization using automatic discrete adjoints in Dedalus. *Left*: We seek to optimize a functional J of the solution $\mathbf{X}$ of a PDE (BVP, EVP, or IVP) over parameters $\mathbf{p}$. *Top Right*: Dedalus already enables solving general PDEs with fast spectral methods with operator graphs representing the PDE (forward pass). *Bottom Right*: We have now automated the adjoint (backward pass) by reversing the high-level solver control flow and implementing efficient vector-Jacobian products (VJPs) on the operator graphs, akin to reverse-mode automatic differentiation. The result is an optimally fast computation of the discrete model gradient.

Results

To demonstrate the simplicity and flexibility of our approach to computing discrete adjoints for sparse spectral solvers, we have created fast implementations using Dedalus of several canonical problems in PDE-based optimization. We have chosen examples using a variety of solver types, illustrating different scenarios in which gradient information is utilized. They are also chosen to cover a range of application areas and domain geometries, showcasing the flexibility of our framework and highlighting its suitability for enhancing a wide range of PDE-based modeling studies. The scripts for these examples, among others, are available on GitHub (*Data, Materials, and Software Availability*).

Parametric Sensitivity and Numerical Continuation. Since adjoints provide efficient access to gradient information, they are an ideal tool for parametric sensitivity analyses where we seek the direct impact of problem parameters on the PDE solution. Here, we consider the process of computing the neutral stability curve for plane Poiseuille flow (56, 57), a canonical fluid dynamics problem that describes pressure-driven flow between two flat plates situated at $y = \pm 1$. In nondimensional variables, the stability problem can be written as an eigenvalue problem involving the velocity and pressure perturbations $\mathbf{u}'$ and p' as

$$\begin{aligned} &\lambda \mathbf{u}' + \mathbf{u}_0 \cdot \nabla \mathbf{u}' + \mathbf{u}' \cdot \nabla \mathbf{u}_0 = -\nabla p' + \frac{1}{\mathrm{Re}} \nabla^2 \mathbf{u}', \\ &\nabla \cdot \mathbf{u}' = 0, \\ &\mathbf{u}'(y = \pm 1) = 0, \end{aligned} \qquad [7]$$

where $\mathbf{u}_0 = (1 - y^2)\hat{\mathbf{e}}_x$ is the base flow and Re is the Reynolds number. The real part of the eigenvalue λ is the growth rate ($\gamma = \Re \lambda$), and the imaginary part gives the oscillation frequency. As the base flow is independent of x, this stability problem is separable over streamwise wavenumbers α. For what follows, we consider only 2D perturbations (with no spanwise z-dependence), and seek to find the sets of parameters (α, Re) such that the maximum growth rate is zero, indicating neutral stability.

To accelerate the process of computing the neutral curve, we utilize discrete model adjoints in Dedalus in two ways. First, we guess a point close to the neutral curve and use the EVP solver and its adjoint to compute the largest growth rate $\gamma_{\max}(\mathrm{Re}, \alpha)$ and its parametric gradient at this point. Using a Newton solver, we can then efficiently find a point on the neutral curve where $\gamma_{\max} = 0$. Second, to construct a guess for a new point on the neutral curve, we use the parametric gradient to find the parametric tangent to the neutral curve, from which a new guess can be estimated. By repeating this process, we can efficiently trace the neutral curve without difficulties continuing around turning points.

Fig. 2 shows the result of this procedure, reproducing the classical stability boundary. Each point indicated on this curve has a growth rate of zero to machine precision and was calculated with approximately five eigenvalue solves. This leads to a very efficient calculation of the stability boundary without requiring a multidimensional grid search. While this equation only contains two parameters, the adjoint method allows quick calculations of eigenvalue sensitivities with respect to potentially many more. Furthermore, the adjoint implementation reuses the LU decomposition from the forward problem, providing significant computational savings over finite differences, which would require new forward solves and factorizations.

Nonlinear Optimization. Propagating sensitivities through nonlinear evolutionary equations is possible with adjoint looping, where the backward pass corresponds to a reverse-time integration using the time-dependent linearization of the forward model. This technique allows optimizing, e.g., final-time functionals with respect to equation parameters and initial conditions. To demonstrate the capabilities of automatic differentiation in Dedalus as a tool for nonlinear optimization, we consider the problem of optimizing dynamo action in a ball with insulating boundary conditions. We follow Chen et al. (58), which utilizes

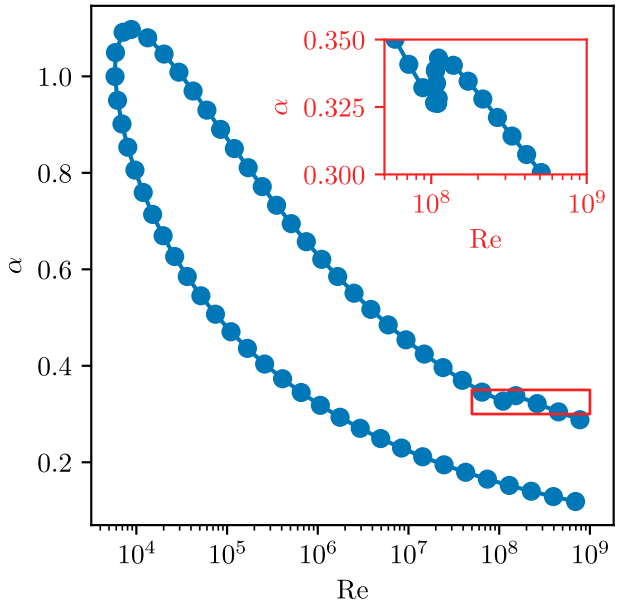

Fig. 2. Neutral stability curve for plane Poiseuille flow obtained with adjoint-based eigenvalue sensitivities and numerical continuation. Each point on the curve has a growth rate of zero to machine precision. The parameters are the horizontal wavenumber α and the base-flow Reynolds number Re.

the optimization procedure established in ref. 59 for Cartesian domains. This is an important and challenging example of using nonlinear optimization to assess the stability of fluid flows [see the review (60) and references therein].

In the astrophysical context, dynamo theory deals with the growth of magnetic fields via the motion of a conducting fluid—an important process in planets, stars, accretion disks, and galaxies (for a review, see ref. 61). This problem is governed by the induction equation,

$$\begin{aligned} &\frac{\partial \mathbf{B}}{\partial t} - \eta \nabla^2 \mathbf{B} = \nabla \times (\mathbf{u} \times \mathbf{B}), \\ &\nabla \cdot \mathbf{B} = 0, \end{aligned} \qquad [8]$$

where $\mathbf{B}(t)$ is the magnetic field, $\mathbf{u}$ is the fluid velocity, and η is the magnetic diffusivity. Although this is a linear evolution equation for $\mathbf{B}(t)$ when the velocity field is fixed, it is nonlinear as a joint optimization over $\mathbf{u}$ and $\mathbf{B}(0)$, and obtaining growing solutions can be a highly nontrivial and subtle task.

Scaling length with the domain radius R, velocity with ΩR for a prescribed vorticity scale Ω, time with R^2/η, and magnetic field with an arbitrary strength B, the induction equation includes a single nondimensional parameter, the vorticity-based magnetic Reynolds number $\mathrm{Rm} \equiv R^2\Omega/\eta$. In order to ensure the magnetic field remains solenoidal, we formulate the problem using a magnetic vector potential $\mathbf{A}(t)$, where $\mathbf{B} = \nabla \times \mathbf{A}$, as

$$\begin{aligned} &\frac{\partial \mathbf{A}}{\partial t} - \nabla^2 \mathbf{A} + \nabla \psi = \mathrm{Rm}\, \mathbf{u} \times \mathbf{B}, \\ &\nabla \cdot \mathbf{A} = 0. \end{aligned} \qquad [9]$$

These equations include a scalar field ψ that is fixed by the Coulomb gauge condition. Since the temporal eigenmodes of $\mathbf{B}$ correspond to temporal eigenmodes of $\mathbf{A}$ with the same growth rate, optimizing the growth of $\mathbf{A}$ beyond initial transients will yield the same optimal velocity field $\mathbf{u}$ as directly optimizing the growth of $\mathbf{B}$.

Defining $\|\mathbf{X}\|^2 = (1/V)\int \mathbf{X} \cdot \mathbf{X}\, \mathrm{d}V$ and following ref. 58, we formulate the optimization problem as maximizing

$$J = \log \|\mathbf{A}(T)\|^2 \qquad [10]$$

with constraints $\|\mathbf{A}(0)\| = 1$ and $\|\boldsymbol{\omega}\| = 1$, where $\boldsymbol{\omega} = \nabla \times \mathbf{u}$ is the vorticity field. Note that the norm of the vorticity, rather than the velocity, is fixed due to the results of ref. 62. We specify vacuum boundary conditions for the magnetic field, which can be written in terms of the spherical harmonic decomposition of $\mathbf{A}$ as

$$\frac{\partial \mathbf{A}_{\ell,\sigma}}{\partial r} + \frac{\ell + 1 + \sigma}{r} \mathbf{A}_{\ell,\sigma} = 0 \quad \text{at } r = 1, \qquad [11]$$

where ℓ is the spherical harmonic degree and we have used the regularity component index $\sigma \in \{-1, 1, 0\}$ (36). Since the norm of $\boldsymbol{\omega}$ is constrained, we obtain $\mathbf{u}$ from $\boldsymbol{\omega}$ by solving the auxiliary LBVP

$$\begin{aligned} &\nabla \times (\nabla \times \mathbf{u}) + \nabla \chi = \nabla \times \boldsymbol{\omega}, \\ &\nabla \cdot \mathbf{u} = 0, \end{aligned} \qquad [12]$$

together with no-slip conditions on $\mathbf{u}$. This ensures that the velocity field remains divergence-free and satisfies appropriate boundary conditions.

We utilize the Pymanopt (63) library to perform the optimization on the product manifold $(\mathbf{A}(0), \boldsymbol{\omega}) \in \Pi = V_{\mathbf{W}} \times V_{\mathbf{W}}$, where $V_{\mathbf{W}} = \left\{\mathbf{x} \mid \mathbf{x}^T \mathbf{W} \mathbf{x} = 1\right\}$ is a generalized Stiefel manifold with a weight matrix $\mathbf{W}$ corresponding to the discretized L^2 norm. Through this formulation, the internal Pymanopt solver uses a Riemannian conjugate gradient method with line search (64), resulting in rapid convergence and simple enforcement of the norm constraints.

The results of the optimization procedure are shown in Fig. 3. From the time series, we see that for all magnetic Reynolds numbers there is a short period of transient growth (up to $t \approx 0.1$), followed by an exponential period in which only the mode with the largest growth rate remains. For $\mathrm{Rm} \lesssim 64.45$, the flow is stable and this growth rate is negative, whereas for $\mathrm{Rm} \gtrsim 64.45$ the growth rate is positive. For $\mathrm{Rm} \approx 64.45$ the growth rate is approximately zero, indicating that this is the critical magnetic Reynolds number. The figure also shows the streamlines and field lines of the optimal velocity and magnetic fields, respectively. We see that the flow is mainly concentrated near the center of the ball, where a large increase in flow velocity occurs with a twisting motion. These results are all in close agreement with ref. 58.

To highlight the flexibility of our method, we also perform the optimization problem of maximizing

$$J = \log \|\mathbf{B}(T)\|^2 \qquad [13]$$

subject to $\|\mathbf{B}(0)\|_2 = 1$. This is done by solving the induction equation directly for $\mathbf{B}$. Fig. 3 shows the resulting magnetic energy under this optimization compared to the initial optimization using $\mathbf{A}$. As expected, both energies show the same growth rates after the initial transient. However, the amount of transient growth is notably different, with the magnetic-energy-based optimization leading to substantially more absolute growth. This example emphasizes the flexibility of our automated adjoint capabilities for spectral solvers, which allow users to easily examine problems from different viewpoints using different equation formulations.

Fig. 3. Optimal kinematic dynamo solutions in the ball from nonlinear optimization via adjoint looping, following Chen et al. (65). *Top Left*: Time series showing the result of optimizing the vector potential norm at different magnetic Reynolds numbers. *Bottom Left*: Time series comparing optimization of the magnetic field norm vs. the vector potential norm. *Middle*: Streamlines of the optimal velocity field for Rm = 64.45. *Right*: Field lines of the optimal magnetic field at $t = 1$ for Rm = 64.45.

Resolvent Analysis. Resolvent analysis is a powerful modal analysis technique (66), whose applications to turbulence were pioneered by McKeon and Sharma (67). In their setup, the flow field is decomposed into a stationary mean flow $\bar{\mathbf{u}}$ and fluctuations $\mathbf{u}'$, such that $\mathbf{u} = \bar{\mathbf{u}} + \mathbf{u}'$. Substituting this into the Navier–Stokes equations and Fourier transforming in time results in the following coupled systems of equations:

$$i\omega\hat{\mathbf{u}}' + \bar{\mathbf{u}}\cdot\nabla\hat{\mathbf{u}}' + \hat{\mathbf{u}}'\cdot\nabla\bar{\mathbf{u}} + \nabla\hat{p} - \frac{1}{\mathrm{Re}}\nabla^2\hat{\mathbf{u}}' = \mathcal{F}[\mathbf{u}'\cdot\nabla\mathbf{u}'](\omega),$$
$$\nabla\cdot\hat{\mathbf{u}}' = 0, \qquad [14]$$

and

$$\bar{\mathbf{u}}\cdot\nabla\bar{\mathbf{u}} + \nabla\bar{p} - \frac{1}{\mathrm{Re}}\nabla^2\bar{\mathbf{u}} = \mathcal{F}[\mathbf{u}'\cdot\nabla\mathbf{u}'](0),$$
$$\nabla\cdot\bar{\mathbf{u}} = 0, \qquad [15]$$

where we denote the Fourier transform at frequency ω as $\mathcal{F}[\cdot](\omega)$. Eq. **14** shows that fluctuations at frequency ω are driven by triadic interactions, represented by the right-hand side. In turn, Eq. **15** shows that the zero-frequency component of these triadic interactions sustains the mean flow.

Instead of solving the fully coupled system, mean flow data can be obtained from simulations or experiments. In doing so, Eq. **14** can be reduced to the form

$$(i\omega\mathbf{M} + \mathbf{L})\hat{\mathbf{u}}' = \hat{\mathbf{f}}, \qquad [16]$$

where $\mathbf{M}$ is a mass matrix, $\mathbf{L}$ is a linear operator, and the triadic interactions are written as a forcing term $\hat{\mathbf{f}}$.

Resolvent analysis treats $\hat{\mathbf{f}}$ as a generic forcing term arising from turbulence and examines Eq. **16** as an input–output system in which a transfer function $\mathcal{H} = (i\omega\mathbf{M}+\mathbf{L})^{-1}$ maps a forcing $\hat{\mathbf{f}}$ to its driven response $\hat{\mathbf{u}}'$. This transfer function $\mathcal{H}$ is known as the resolvent. Of interest are the leading singular values and vectors of this operator, i.e., $(\sigma_i, \hat{\mathbf{u}}_i', \hat{\mathbf{f}}_i)$ such that $\mathcal{H}\hat{\mathbf{f}}_i = \sigma_i\hat{\mathbf{u}}_i'$ and the vectors $\hat{\mathbf{u}}_i'$ and $\hat{\mathbf{f}}_i$ are orthonormal under the L^2 (energy) norm. The gains σ_i measure the amount of amplification a particular forcing induces in the flow response. In situations where the largest gain σ_1 is much greater than the next largest σ_2, the flow is deemed low-rank, and a generic forcing $\hat{\mathbf{f}}$ is likely to produce a response that resembles $\hat{\mathbf{u}}_1'$ due to the orthogonality of the singular vectors.

Hence, a resolvent analysis reveals two key insights. First, it systematically identifies the dominant frequencies of turbulent fluctuations, where the gains are maximized. Second, for frequencies at which the flow is found to be low-rank, the leading response mode is expected to resemble coherent structures found in the turbulent flow. Furthermore, by examining the corresponding forcing mode, one can gain important information relevant for flow control (see, e.g., refs. 68 and 69). While we focus here on resolvent analysis for turbulent flows (about the mean flow), it is worth noting that resolvent analysis for steady flows (about fixed points) is also an important concept in nonmodal stability analysis (1, 70), predating its use in turbulence, and serves as the driven counterpart to transient growth analysis.

Using our automatic adjoint routines in Dedalus, we reproduce the resolvent analysis for turbulent pipe flow following ref. 67. Since the mean flow is obtained by averaging over ϕ and z, it depends only on the radial coordinate, allowing us to solve for the forcing and response independently for each azimuthal wavenumber m and streamwise wavenumber k. We use Dedalus to compute the action of both $\mathcal{H}$ and $\mathcal{H}^\dagger$ for given m and k in a cylindrical geometry. This is done using a linear boundary value problem (LBVP) to apply $\mathcal{H}$ via solving the associated matrix pencil, and applying the adjoint via the vector-Jacobian product of the LBVP. Together, these routines are used to compute the leading singular vectors via the SciPy sparse SVD. The mean flow is taken from experimental data at Re = 74,345 (71).

Fig. 4 shows the resolvent results for azimuthal wavenumber $m = 10$ and streamwise wavenumber $k = 1$, in excellent agreement with ref. 67. The figure illustrates that over a wide range of frequencies, the flow is low-rank, with an order-of-magnitude separation between σ_1 and σ_2. The optimal response near the peak gain at $\omega = 0.5$ is concentrated near the wall

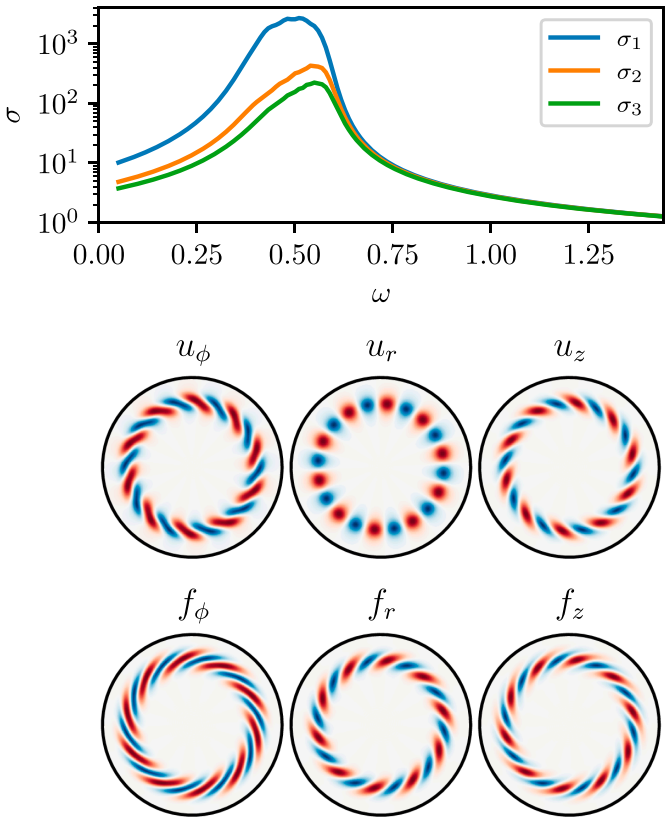

Fig. 4. Resolvent analysis for turbulent pipe flow at Re = 74,345, following McKeon and Sharma (67), for axial wavenumber $k = 1$ and azimuthal wavenumber $m = 10$. *Top*: Leading gains (singular values) as a function of frequency, showing the optimal response at $\omega \approx 0.5$. *Bottom*: Optimal forcing and response modes, showing structures similar to those observed in experimental fluctuations.

and resembles large-scale motions observed in turbulent pipe experiments. This example demonstrates the ease of setting up and performing a resolvent analysis using a differentiable spectral code. Since many turbulent experimental configurations—such as pipe flow, channel flow, and Taylor–Couette flow—can be modeled spectrally, there is a clear advantage to complementing these experiments with resolvent analysis. In particular, we envision that resolvent analyses for fluids not modeled by the incompressible Navier–Stokes equations (e.g., compressible, conducting, non-Newtonian, or viscoelastic fluids) will be made more accessible by these developments, which significantly ease the burden of implementation.

Phase Reduction Analysis. Phase-reduction analysis is a technique with origins in mathematical biology and neuroscience that is used to study phase dynamics and synchronization in oscillating systems (72, 73). More recently, it has gained traction in the fluid mechanics community (74), where it has been applied to assess the synchronization properties of fluid–structure interactions (75), thermoacoustic instability (76), and to design flow actuation strategies in aeronautical engineering applications (77, 78).

For a stable limit-cycle solution with period T, the notion of phase can be defined as follows. The phase θ is first defined for states on the limit cycle as $2\pi t/T$ for $t \in [0, T]$. In other words, for a state on the limit cycle, the phase variable $\theta \in [0, 2\pi]$ assigns a scalar label to each state, and satisfies the simple ODE $\dot{\theta} = 2\pi/T$ on the cycle. This definition can be extended to states in the vicinity of the limit cycle by using the fact that the cycle is stable, and therefore any small perturbation returns to it. Hence, for a state near the limit cycle, we define its phase as the phase of the point on the limit cycle whose trajectory it matches asymptotically as $t \to \infty$. Under this definition, the phase dynamics near the limit cycle are governed by the ODE

$$\dot{\theta} = \frac{2\pi}{T} + \mathbf{z}(\theta) \cdot \mathbf{h}(\theta), \qquad [17]$$

where $\mathbf{h}(\theta)$ represents a small perturbation to the system and $\mathbf{z}(\theta)$ is the phase sensitivity function, whose computation is the central task in a phase-reduction analysis. By obtaining $\mathbf{z}$, the phase response can be determined independently of the specific form of the perturbation, making this a powerful technique for analyzing the phase and synchronization dynamics of oscillators.

As an example of how adjoints enable spectral computations for phase-reduction analysis, we consider the FitzHugh-Nagumo model (79, 80):

$$\begin{aligned} \frac{\mathrm{d}u}{\mathrm{d}t} &= u - \frac{u^3}{3} - v + I, \\ \frac{\mathrm{d}v}{\mathrm{d}t} &= \epsilon(u + a - bv), \end{aligned} \qquad [18]$$

a simplified version of the Hodgkin–Huxley model (81). These equations model the firing dynamics of a neuron, with membrane potential u and recovery variable v. The parameters include ϵ (which sets the timescale separation between u and v), the input current I, and constants a and b describing the activation dynamics. For $I \neq 0$, the system exhibits slow-fast repetitive firing for a range of parameter values, yielding a stable limit-cycle solution.

Of interest is the phase response of the dynamics about this limit cycle. This includes how perturbations can delay or advance the phase of the solution, synchronization of neurons to external stimuli (82), collective synchronization of coupled neurons (83), and synchronization of oscillators to common white noise (84). The key to understanding all of these phenomena lies in computing the phase sensitivity function, which can be efficiently obtained via adjoint methods (see ref. 73, for example). This connection between the adjoint problem and the phase sensitivity function is essential for efficiently conducting a phase-reduction analysis, particularly in PDE systems, as recently demonstrated for fluid flows (85).

To compute the phase sensitivity function using Dedalus, we first compute the limit-cycle solution (u_0, v_0). To do this, we discretize Eq. **18** in time using a Fourier basis, and solve it as a nonlinear boundary value problem (NLBVP) to obtain a spectrally accurate periodic solution. Floquet theory then gives that perturbations to this limit cycle (u', v') satisfy the eigenvalue problem

$$\begin{aligned} \frac{\mathrm{d}u'}{\mathrm{d}t} + \lambda u' &= u' - u_0^2 u' - v', \\ \frac{\mathrm{d}v'}{\mathrm{d}t} + \lambda v' &= \epsilon(u' - bv'), \end{aligned} \qquad [19]$$

where λ is the eigenvalue and where u' and v' are again discretized using Fourier series in t [the Floquet–Fourier–Hill method (86)]. Since the system is autonomous, it has a neutral eigenvalue $\lambda = 0$ with corresponding eigenvector $(\dot{u}_0, \dot{v}_0)$ representing phase shifts along the limit cycle. The phase sensitivity function, which allows the projection of dynamics onto this phase shift, is obtained

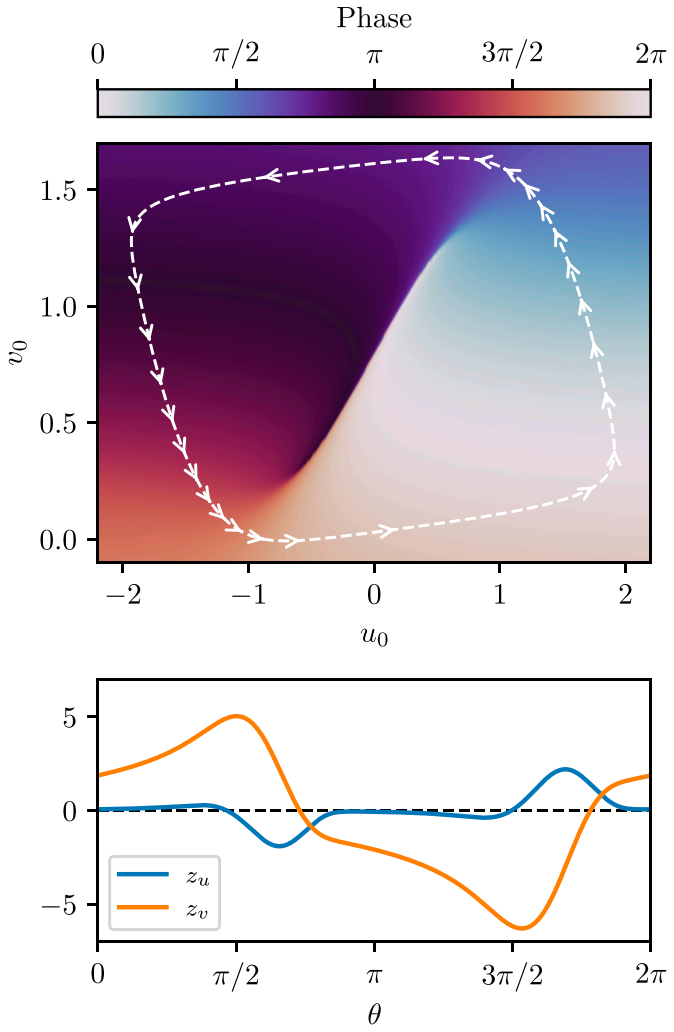

Fig. 5. Phase sensitivity analysis of the FitzHugh-Nagumo model via discrete eigenvalue problem adjoints. *Top*: Limit cycle and explicitly computed phase function. Arrows on the limit cycle are equispaced in time, illustrating the slow-fast dynamics of the oscillator. *Bottom*: The phase sensitivity components measuring the gradient of the phase function along the limit cycle.

as the corresponding adjoint eigenvector, normalized such that $\langle \mathbf{z}, (\dot{u}_0, \dot{v}_0)^T \rangle = 1$. Thus, the phase sensitivity function can be computed in Dedalus simply by solving the adjoint of Eq. **19**.

The limit cycle, the explicitly computed phase function, and the phase sensitivity as computed with our adjoint method are shown in Fig. 5 for parameters $a = 0.7$, $b = 0.8$, $\epsilon = 0.08$, and $I = 0.8$ and demonstrate excellent agreement with ref. 87. This approach can be easily adapted to other equation sets, including PDEs, providing a systematic route to phase-reduction analysis for a wide range of oscillators.

Discussion

We have presented a framework for producing fast adjoint solvers for sparse spectral methods and have demonstrated its implementation in the Dedalus library. Leveraging the structure of the underlying sparse algorithms, we have developed a hybrid approach in which adjoints are explicitly implemented for fundamental operations and composed via a high-level reverse-mode AD system. This enables the efficient evaluation of vector-Jacobian products for arbitrary nonlinear expressions based on their graph representations in Dedalus. By individually handling procedures such as linear system solves, spectral transforms, timestepping routines, eigenvalue solvers, and Newton solvers, we can quickly and reliably compute adjoints of PDE workflows that would otherwise be inefficiently traced by an AD compiler. This implementation is easily extendable by end users, who can add custom differentiable operators as simple Python classes.

Our implementation reuses data structures from the forward solver—such as matrix decompositions and transform plans—so that the resulting adjoint solver is both computationally and memory efficient. It fully supports all domains, problem types, high-order integrators, and parallelization strategies available in Dedalus, a distinct advantage over currently available differentiable solvers, e.g., JAX-CFD. This flexibility enables users to obtain adjoint information for general PDEs directly from their forward model implementations in Dedalus, without requiring additional libraries or restructuring of the original code.

To demonstrate the capabilities of this approach, we have reproduced canonical examples using adjoint methods from classical fluid mechanics, dynamo theory, modal analysis of turbulence, and mathematical neuroscience. These examples span a range of problem types and geometries, showcasing the versatility and advantages of a differentiable spectral framework. Specifically, our examples illustrate how such a code can be used for parametric sensitivity analysis, numerical continuation, nonlinear optimization, nonmodal stability theory, and phase sensitivity analysis. These methods are applicable across scientific disciplines and utilize gradient information in diverse ways. By extending open-source differentiable solvers to encompass modern spectral methods, this work enables adjoint-based studies in domains where spectral approaches are essential for accuracy and efficiency.

In this work, we have focused on first-order model derivatives and coupling to gradient-based optimizers. A natural next step is to automate the computation of higher-order derivatives for more general optimization routines, e.g., ref. 88. This would enable techniques such as Newton-type methods for optimization as well as uncertainty quantification and Bayesian inference, which can benefit from efficient Hessian-vector products for posterior sampling (see e.g., refs. 89 and 90). Additionally, it is now possible to combine Dedalus models with machine learning libraries by manually linking gradient computations. Creating a dedicated interface to simplify this integration—similar to the interface between Firedrake and PyTorch (91)—would enable seamless end-to-end model-based training for a wide range of applications.

Finally, an important future direction is the integration of adjoints with other ongoing extensions to Dedalus, including new bases for additional geometries and optimizations for graphics and tensor processing units. Key to this integration is the foundation we have established in rendering the code differentiable via high-level abstractions and modular design, rather than low-level implementations tied to a specific AD compiler. This design will allow adjoint capabilities to be maintained and extended efficiently across emerging platforms and optimization ecosystems.

Materials and Methods

Solving the adjoint state equation, Eq. **5**, requires adjoint linear-system solves–e.g., for sparse matrices A_i–and the evaluation of vector-Jacobian products for the right-hand-side (RHS) terms $J(\mathbf{X}, \mathbf{p})$ and $\mathbf{F}(\mathbf{X}, \mathbf{p})$. Here, we outline the approach

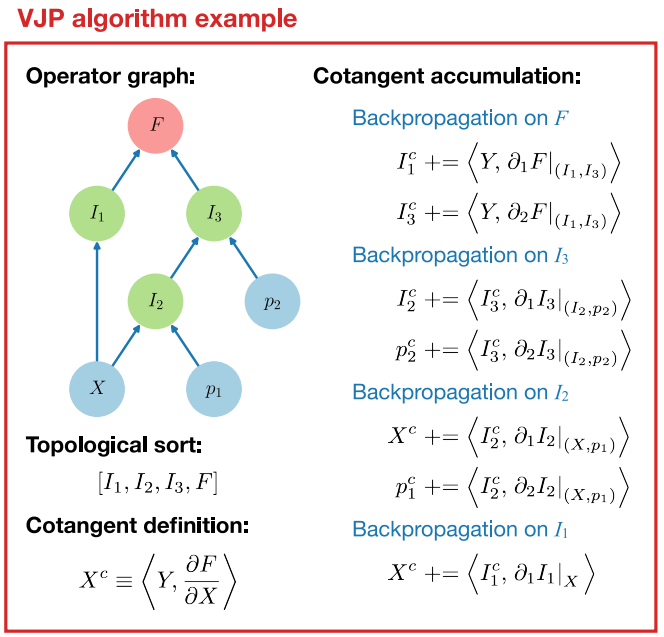

Fig. 6. Example of evaluating a VJP via backpropagation through a symbolic operator graph. The graph is first forward-evaluated to compute the primal values of all intermediate operators I_j. This evaluation produces a topological sort of the graph. Cotangents for each intermediate result and each operand (X, p_j) are accumulated by applying VJPs for each operation in the reverse order of the topological sort. Each VJP uses the cotangent of the respective operator and is evaluated using the primal values.

used to efficiently automate these processes in Dedalus. For details on how different problem types map to this structure, see *SI Appendix*.

Linear System Solves. In each forward iteration, linear systems are solved using fast direct solvers, typically via sparse LU factorizations of A_j. The corresponding adjoint solves are performed by solving the adjoints of the LU factors of A_j, without requiring additional factorizations.

Nonlinear Operator Graphs. Nonlinear terms are represented in Dedalus as directed acyclic graphs (DAGs) of operators acting on the fields and parameters of the PDE. These graphs are evaluated via depth-first recursion, which triggers spectral transforms as needed to compute intermediate operators.

Forward-mode (tangent) sensitivity propagation through an operator graph is implemented as a matrix-free Jacobian-vector product (JVP) evaluated concurrently with the primal graph. This only requires each operator to provide its discrete derivative.

Reverse-mode (adjoint/cotangent) sensitivity propagation is implemented as a matrix-free vector-Jacobian product (VJP) utilizing the existing DAG operator structure. During the forward evaluation of the operator graph, a tape records the evaluation order of intermediate operators, forming a topological sort of the DAG. The VJP is computed by propagating the cotangents backward through this recorded evaluation order and accumulating at the root nodes. This approach is equivalent to reverse-mode automatic differentiation but is performed in a reliably memory- and compute-efficient manner by leveraging the high-level structure of the forward operator DAG.

Fig. 6 illustrates this process for an explicit example graph structure. The backpropagation steps for linear operators are performed by directly applying the adjoints of the sparse operator matrices to the incoming cotangents. The backpropagation steps for local nonlinear operators, such as multiplication, are evaluated on a collocation grid using adjoint transforms, where the operator Jacobian is diagonal (separable over grid points). The cotangents for all variables are initialized to zero at the beginning of an adjoint step, and the contributions from VJPs of J and $\mathbf{F}$ are accumulated in-place to produce the full derivatives in Eq. **5**.

Spectral Transforms. During the evaluation of nonlinear operator graphs, spectral transforms are employed to efficiently apply differential operators to field coefficients in their sparse spectral representations, while nonlinearities are evaluated on a collocation grid. This pseudospectral approach is substantially faster than either pure-collocation or pure-spectral approaches when fast transforms are available.

When evaluating operator VJPs, the adjoint of each spectral transform must therefore be applied. To automate this process, we have implemented specialized "adjoint field" classes that represent cotangent quantities and overload transformation behaviors to automatically apply the appropriate adjoint transforms.

Let $\mathbf{f}$ denote a field's values on the collocation grid and $\hat{\mathbf{f}}$ its spectral coefficients. Let $\mathbf{T}$ denote the forward transform, such that $\hat{\mathbf{f}} = \mathbf{T}\mathbf{f}$ and $\mathbf{f} = \mathbf{T}^{-1}\hat{\mathbf{f}}$. Denoting an adjoint field as $\mathbf{f}^\dagger$, we require that $\langle \mathbf{f}^\dagger, \mathbf{f} \rangle = \langle \hat{\mathbf{f}}^\dagger, \hat{\mathbf{f}} \rangle$. Using the above identities, we obtain the relationships

$$\hat{\mathbf{f}}^\dagger = \mathbf{T}^{-\dagger}\mathbf{f}^\dagger, \quad \mathbf{f}^\dagger = \mathbf{T}^\dagger\hat{\mathbf{f}}^\dagger, \qquad [20]$$

showing that the forward transform of an adjoint field is the adjoint of the original backward transform, and vice versa.

For transforms implemented directly as matrix multiplications, the adjoints are computed using the Hermitian transpose of the matrix. For fast transforms (e.g., FFTs), the adjoint is implemented using a corresponding fast transform, depending on the transform type.

Supported Operators. Many common operators are implemented in the Dedalus codebase, including differential operators, basis conversions, linear functionals for boundary conditions, and common nonlinear terms. The discrete JVPs and VJPs for linear operators generally use their matrix forms, and we have manually defined the JVP and VJP actions for each nonlinear operator. The correctness of these discrete derivatives are verified with unit tests.

Custom operators can be easily added by creating Python classes that inherit from the included operator base classes. For linear operators, only the operator matrix form needs to be specified. For nonlinear operators, the primal and differential actions are simply defined within methods on each custom class.

Data, Materials, and Software Availability. Code for the example problems and instructions for installing all necessary software are available online (92).

ACKNOWLEDGMENTS. We thank Steve Tobias, Jeff Oishi, Geoff Vasil, Daniel Lecoanet, Ben Brown, Rich Kerswell, Colm Caulfield, Patrick Farrell, Florence Marcotte, Paul Mannix, Andre Souza, Greg Wagner, James Maddison, Jacob Page, and Peter Schmid for many helpful discussions. We also thank the reviewers for their valuable comments, which improved the manuscript. This work was undertaken on ARC4, part of the High Performance Computing facilities at the University of Leeds, UK. This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement no. D5S-DLV-786780. This work was supported by a grant from the Simons Foundation (Grant 662962, GF) and a grant from the Simons Foundation International (SSRFA 6826, K.J.B.).

Author affiliations: [a]Department of Applied Mathematics, University of Leeds, Leeds LS2 9JT, United Kingdom; [b]School of Physics and Astronomy, The University of Edinburgh, Edinburgh EH9 3FD, United Kingdom; [c]Department of Mathematics, Massachusetts Institute of Technology, Cambridge, MA 02139; and [d]Center for Computational Astrophysics, Flatiron Institute, New York, NY 10010

1. L. N. Trefethen, A. E. Trefethen, S. C. Reddy, T. A. Driscoll, Hydrodynamic stability without eigenvalues. *Science* **261**, 578–584 (1993).
2. P. J. Schmid, D. S. Henningson, *Stability and Transition in Shear Flows, Applied Mathematical Sciences* (Springer-Verlag, New York, NY, 2001), vol. 142.
3. P. Luchini, A. Bottaro, Adjoint equations in stability analysis. *Annu. Rev. Fluid Mech.* **46**, 493–517 (2014).
4. G. A. Mensah, A. Orchini, J. P. Moeck, Perturbation theory of nonlinear, non-self-adjoint eigenvalue problems: Simple eigenvalues. *J. Sound Vib.* **473**, 115200 (2020).
5. F. X. L. Dimet, O. Talagrand, Variational algorithms for analysis and assimilation of meteorological observations: Theoretical aspects. *Tellus A* **38A**, 97–110 (1986).
6. A. C. Lorenc, Analysis methods for numerical weather prediction. *Q. J. R. Meteorol. Soc.* **112**, 1177–1194 (1986).

7. A. Jameson, *Aerodynamic Shape Optimization, Cambridge Aerospace Series* (Cambridge University Press, 2022), pp. 465–495.
8. L. Magri, Adjoint methods as design tools in thermoacoustics. *Appl. Mech. Rev.* **71**, 020801 (2019).
9. E. J. Paul, I. G. Abel, M. Landreman, W. Dorland, An adjoint method for neoclassical stellarator optimization. *J. Plasma Phys.* **85**, 795850501 (2019).
10. M. Landreman *et al*., SIMSOPT: A flexible framework for stellarator optimization. *J. Open Sour. Softw.* **6**, 3525 (2021).
11. R. E. Plessix, A review of the adjoint-state method for computing the gradient of a functional with geophysical applications. *Geophys. J. Int.* **167**, 495–503 (2006).
12. M. B. Giles, N. A. Pierce, An introduction to the adjoint approach to design. *Flow, Turbul. Combust.* **65**, 393–415 (2000).
13. Y. Ma, V. Dixit, M. J. Innes, X. Guo, C. Rackauckas, "A comparison of automatic differentiation and continuous sensitivity analysis for derivatives of differential equation solutions" in *2021 IEEE High Performance Extreme Computing Conference (HPEC)* (IEEE, 2021), pp. 1–9.
14. F. Palacios *et al*., *Stanford University Unstructured (SU^2): An Open-Source Integrated Computational Environment for Multi-Physics Simulation and Design* (Stanford University Unstructured, 2013).
15. A. Logg, G. Wells, K. A. Mardal, *Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book* (Springer, 2011), vol. 84.
16. D. A. Ham *et al*., *Firedrake User Manual* (Imperial College London and University of Oxford and Baylor University and University of Washington, ed. 1, 2023).
17. P. Holl, N. Thuerey, "*Phi*$_{\text{flow}}$ (PhiFlow): Differentiable simulations for pytorch, tensorflow and jax" in *International Conference on Machine Learning*, R. Salakhutdinov *et al*., Eds. (PMLR, 2024).
18. K. Giannakoglou, E. Papoutsis-Kiachagias, K. Gkaragkounis, *adjointOptimisationFoam, an OpenFOAM-Based Optimisation Tool* (The Parallel CFD & Optimization Unit, School of Mechanical Engineering, National Technical University of Athens, 2019).
19. F. Koehler, S. Niedermayr, R. Westermann, N. Thuerey, APEBench: A benchmark for autoregressive neural emulators of PDEs. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **38**, 120252–120310 (2024).
20. D. A. Bezgin, A. B. Buhendwa, N. A. Adams, JAX-Fluids: A fully-differentiable high-order computational fluid dynamics solver for compressible two-phase flows. *Comput. Phys. Commun.* **282**, 108527 (2023).
21. D. A. Bezgin, A. B. Buhendwa, N. A. Adams, JAX-Fluids 2.0: Towards HPC for differentiable CFD of compressible two-phase flows. *Comput. Phys. Commun.* **308**, 109433 (2025).
22. D. Kochkov *et al*., Machine learning–accelerated computational fluid dynamics. *Proc. Natl. Acad. Sci. U.S.A.* **118**, e2101784118 (2021).
23. G. Dresdner *et al*., Learning to correct spectral methods for simulating turbulent flows (2023). *https://openreview.net/forum?id=wNBARGxoJn*. Accessed 23 October 2025.
24. H. Ranocha *et al*., Adaptive numerical simulations with trixi.jl: A case study of julia for scientific computing. *Proc. JuliaCon Conf.* **1**, 77 (2022).
25. C. Rackauckas *et al*., Universal differential equations for scientific machine learning. arXiv [Preprint] (2021). https://arxiv.org/abs/2001.04385 (Accessed 23 October 2025).
26. M. S. Alnaes, A. Logg, K. B. Ølgaard, M. E. Rognes, G. N. Wells, Unified form language: A domain-specific language for weak formulations of partial differential equations. *ACM Trans. Math. Softw.* **40**, 1–37 (2014).
27. P. E. Farrell, D. A. Ham, S. W. Funke, M. E. Rognes, Automated derivation of the adjoint of high-level transient finite element programs. *SIAM J. Sci. Comput.* **35**, C369–C393 (2013).
28. S. K. Mitusch, S. W. Funke, J. S. Dokken, Dolfin-adjoint 2018.1: Automated adjoints for FEniCS and Firedrake. *J. Open Sour. Softw.* **4**, 1292 (2019).
29. Z. Wang, J. B. Estrada, E. M. Arruda, K. Garikipati, Inference of deformation mechanisms and constitutive response of soft material surrogates of biological tissue by full-field characterization and data-driven variational system identification. *J. Mech. Phys. Solids* **153**, 104474 (2021).
30. T. Roy, M. A. Salazar, M. A. de Troya, VA Beck. Worsley, Topology optimization for the design of porous electrodes. *Struct. Multidiscip. Optim.* **65**, 171 (2022).
31. J. P. Boyd, *Chebyshev and Fourier Spectral Methods* (Courier Corporation, 2001).
32. P. Yeung, K. Ravikumar, S. Nichols, R. Uma-Vaideswaran, Gpu-enabled extreme-scale turbulence simulations: Fourier pseudo-spectral algorithms at the exascale using openmp offloading. *Comput. Phys. Commun.* **306**, 109364 (2025).
33. N. P. Wedi, M. Hamrud, G. Mozdzynski, A fast spherical harmonics transform for global NWP and climate models. *Mon. Weather Rev.* **141**, 3450–3461 (2013).
34. S. Olver, A. Townsend, A fast and well-conditioned spectral method. *Siam Rev.* **55**, 462–489 (2013).
35. G. M. Vasil *et al*., Tensor calculus in polar coordinates using Jacobi polynomials. *J. Comput. Phys.* **325**, 53–73 (2016).
36. G. M. Vasil, D. Lecoanet, K. J. Burns, J. S. Oishi, B. P. Brown, Tensor calculus in spherical coordinates using Jacobi polynomials. Part-I: Mathematical analysis and derivations. *J. Comput. Phys. X* **3**, 100013 (2019).
37. J. A. Jackson *et al*., Scaling behaviour and control of nuclear wrinkling. *Nat. Phys.* **19**, 1927–1935 (2023).
38. E. H. Anders *et al*., The photometric variability of massive stars due to gravity waves excited by core convection. *Nat. Astron.* **7**, 1228–1234 (2023).
39. G. M. Vasil *et al*., The solar dynamo begins near the surface. *Nature* **629**, 769–772 (2024).
40. K. J. Burns, G. M. Vasil, J. S. Oishi, D. Lecoanet, B. P. Brown, Dedalus: A flexible framework for numerical simulations with spectral methods. *Phys. Rev. Res.* **2**, 023068 (2020).
41. N. Boumal, B. Mishra, P. A. Absil, R. Sepulchre, Manopt, a matlab toolbox for optimization on manifolds. *J. Mach. Learn. Res.* **15**, 1455–1459 (2014).
42. H. Zhang, E. M. Constantinescu, B. F. Smith, PETSc TSAdjoint: A discrete adjoint ODE solver for first-order and second-order sensitivity analysis. *SIAM J. Sci. Comput.* **44**, C1–C24 (2022).
43. A. Paszke *et al*., PyTorch: An imperative style, high-performance deep learning library. *Adv. Neural Inf. Process. Syst.* **32**, 8026–8037 (2019).
44. D. Lecoanet, R. R. Kerswell, Connection between nonlinear energy optimization and instantons. *Phys. Rev. E* **97**, 012212 (2018).
45. L. O'Connor *et al*., Iterative methods for Navier-Stokes inverse problems. *Phys. Rev. E* **109**, 045108 (2024).
46. C. S. Skene, F. Marcotte, S. M. Tobias, On nonlinear transitions, minimal seeds and exact solutions for the geodynamo. *J. Fluid Mech.* **1021**, A37 (2025).
47. P. M. Mannix, C. S. Skene, D. Auroux, F. Marcotte, A robust, discrete-gradient descent procedure for optimisation with time-dependent PDE and norm constraints. *SMAI J. Comput. Math.* **10**, 1–28 (2024).
48. W. S. Moses *et al*., "Reverse-mode automatic differentiation and optimization of GPU kernels via enzyme" in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21* (Association for Computing Machinery, New York, NY, USA, 2021).
49. J. Bradbury *et al*., JAX: Composable transformations of Python+NumPy programs. GitHub. http://github.com/google/jax. Accessed 23 October 2025.
50. L. Hascoet, V. Pascual, The Tapenade Automatic Differentiation tool: principles, model, and specification. *ACM Trans. Math. Softw.* **39**, 1–43 (2013).
51. D. Lecoanet, G. M. Vasil, K. J. Burns, B. P. Brown, J. S. Oishi, Tensor calculus in spherical coordinates using Jacobi polynomials. Part-II: Implementation and examples. *J. Comput. Phys. X* **3**, 100012 (2019).
52. S. Olver, A. Townsend, G. Vasil, A sparse spectral method on triangles. *SIAM J. Sci. Comput.* **41**, A3728–A3756 (2019).
53. A. C. Ellison, K. Julien, G. M. Vasil, A gyroscopic polynomial basis in the sphere. *J. Comput. Phys.* **460**, 111170 (2022).
54. K. J. Burns, D. Fortunato, K. Julien, G. M. Vasil, Corner cases of the tau method: Symmetrically imposing boundary conditions on hypercubes. arXiv [Preprint] (2024). http://arxiv.org/abs/2211.17259 (Accessed 24 June 2024).
55. S. G. Johnson, Notes on adjoint methods for 18.335 (2021). https://math.mit.edu/~stevenj/18.336/adjoint.pdf. Accessed 23 October 2025.
56. W. Tollmien, Über die entstehung der turbulenz. 1. mitteilung. *Nachr. Gesellsch. Wiss. Göttingen Math. Phys. Kl.* **1929**, 21–44 (1928).
57. S. A. Orszag, Accurate solution of the Orr-Sommerfeld stability equation. *J. Fluid Mech.* **50**, 689–703 (1971).
58. L. Chen *et al*., The optimal kinematic dynamo driven by steady flows in a sphere. *J. Fluid Mech.* **839**, 1–32 (2018).
59. A. P. Willis, Optimization of the magnetic dynamo. *Phys. Rev. Lett.* **109**, 251101 (2012).
60. R. Kerswell, Nonlinear nonmodal stability theory. *Annu. Rev. Fluid Mech.* **50**, 319–345 (2018).
61. S. M. Tobias, The turbulent dynamo. *J. Fluid Mech.* **912**, P1 (2021).
62. M. Proctor, Energy requirement for a working dynamo. *Geophys. Astrophys. Fluid Dyn.* **109**, 611–614 (2015).
63. J. Townsend, N. Koep, S. Weichwald, Pymanopt: A python toolbox for optimization on manifolds using automatic differentiation. *J. Mach. Learn. Res.* **17**, 1–5 (2016).
64. H. Sato, Riemannian conjugate gradient methods: General framework and specific algorithms with convergence analyses. *SIAM J. Optim.* **32**, 2690–2717 (2022).
65. L. Chen, W. Herreman, A. Jackson, Optimal dynamo action by steady flows confined to a cube. *J. Fluid Mech.* **783**, 23–45 (2015).
66. K. Taira *et al*., Modal analysis of fluid flows: An overview. *AIAA J.* **55**, 4013–4041 (2017).
67. B. J. McKeon, A. S. Sharma, A critical-layer framework for turbulent pipe flow. *J. Fluid Mech.* **658**, 336–382 (2010).
68. K. Taira *et al*., Modal analysis of fluid flows: Applications and outlook. *AIAA J.* **58**, 998–1022 (2020).
69. L. V. Rolandi, J. H. M. Ribeiro, C. A. Yeh, K. Taira, An invitation to resolvent analysis. *Theoret. Comput. Fluid Dyn.* **38**, 603–639 (2024).
70. P. J. Schmid, Nonmodal stability theory. *Annu. Rev. Fluid Mech.* **39**, 129–162 (2007).
71. B. J. McKeon, J. Li, W. Jiang, J. F. Morrison, A. J. Smits, Further observations on the mean velocity distribution in fully developed pipe flow. *J. Fluid Mech.* **501**, 135–147 (2004).
72. A. T. Winfree, *The Geometry of Biological Time, Interdisciplinary Applied Mathematics* (Springer-Verlag, New York, NY, 2001), vol. 12.
73. G. B. Ermentrout, D. Terman, *Mathematical Foundations of Neuroscience, Interdisciplinary Applied Mathematics* (Springer, New York, NY, 2010), vol. 35.
74. K. Taira, H. Nakao, Phase-response analysis of synchronization for periodic flows. *J. Fluid Mech.* **846**, R2 (2018).
75. I. A. Loe, H. Nakao, Y. Jimbo, K. Kotani, Phase-reduction for synchronization of oscillating flow by perturbation on surrounding structure. *J. Fluid Mech.* **911**, R2 (2021).
76. C. S. Skene, K. Taira, Phase-reduction analysis of periodic thermoacoustic oscillations in a Rijke tube. *J. Fluid Mech.* **933**, A35 (2022).
77. A. G. Nair, K. Taira, B. W. Brunton, S. L. Brunton, Phase-based control of periodic flows. *J. Fluid Mech.* **927**, A30 (2021).
78. V. Godavarthi, Y. Kawamura, K. Taira, Optimal waveform for fast synchronization of airfoil wakes. *J. Fluid Mech.* **976**, R1 (2023).
79. R. Fitzhugh, Impulses and physiological states in theoretical models of nerve membrane. *Biophys J.* **1**, 445–466 (1961).
80. J. Nagumo, S. Arimoto, S. Yoshizawa, An active pulse transmission line simulating nerve axon. *Proc. IRE* **50**, 2061–2070 (1962).
81. A. L. Hodgkin, A. F. Huxley, A quantitative description of membrane current and its application to conduction and excitation in nerve. *J. Physiol.* **117**, 500–544 (1952).
82. Y. Kuramoto, *Chemical Oscillations, Waves, and Turbulence* (Springer-Verlag, Berlin, 1984).
83. T. Ichinomiya, Frequency synchronization in a random oscillator network. *Phys. Rev. E* **70**, 026116 (2004).
84. H. Nakao, K. Arai, Y. Kawamura, Noise-induced synchronization and clustering in ensembles of uncoupled limit-cycle oscillators. *Phys. Rev. Lett.* **98**, 184101 (2007).
85. Y. Kawamura, V. Godavarthi, K. Taira, Adjoint-based phase reduction analysis of incompressible periodic flows. *Phys. Rev. Fluids* **7**, 104401 (2022).
86. G. W. Hill, On the part of the motion of the lunar perigee which is a function of the mean motions of the sun and moon. *Acta Math.* **8**, 1–36 (1886).
87. H. Nakao, Phase reduction approach to synchronisation of nonlinear oscillators. *Contemp. Phys.* **57**, 188–214 (2016).
88. J. R. Maddison, D. N. Goldberg, B. D. Goddard, Automated calculation of higher order partial differential equation constrained derivative information. *SIAM J. Sci. Comput.* **41**, C417–C445 (2019).
89. W. C. Thacker, The role of the Hessian matrix in fitting models to measurements. *J. Geophys. Res. Oceans* **94**, 6177–6196 (1989).

90. T. Isaac, N. Petra, G. Stadler, O. Ghattas, Scalable and efficient algorithms for the propagation of uncertainty from data through inference to prediction for large-scale problems, with application to flow of the Antarctic ice sheet. *J. Comput. Phys.* **296**, 348–368 (2015).
91. N. Bouziani, D. A. Ham, Physics-driven machine learning models coupling pytorch and firedrake. arXiv [Preprint] (2023). https://arxiv.org/abs/2303.06871 (Accessed 23 October 2025).
92. C. S. Skene, K. J. Burns, Dedalus adjoint examples. GitHub. https://github.com/csskene/dedalus_adjoint_examples. Deposited 23 October 2025.